

โครงการ Socket Programming TICKETPASS: Protocol สำหรับระบบจองบัตรคอนเสิร์ต

วีระศักดิ์ ฌนอมกาย หมู่เรียนที่ 1 รหัสนิสิต 6710405508

1. วัตถุประสงค์ของโปรแกรม

โครงการนี้พัฒนา application-layer protocol ชื่อ TICKETPASS สำหรับระบบจองบัตรคอนเสิร์ตที่มีจำนวนจำกัด โดยเน้นแก้ปัญหาการจองบัตรพร้อมกันจากผู้ใช้หลายคน ซึ่งอาจทำให้เกิดการจองบัตรเกินจำนวนจริง หรือที่เรียกว่า *overselling*

วัตถุประสงค์ของระบบมีดังนี้

1. ให้ผู้ใช้ตรวจสอบจำนวนบัตรที่เหลือในแต่ละโซนได้
2. ให้ผู้ใช้ล็อกบัตรไว้ชั่วคราวก่อนยืนยันการจอง
3. ป้องกันการจองบัตรเกินจำนวน แม้มี client หลายเครื่องส่งคำขอพร้อมกัน
4. คินบัตรเข้าสู่ระบบเมื่อผู้ใช้ไม่ยืนยันการจองภายในเวลาที่กำหนด
5. ป้องกันการล็อกบัตรซ้ำจากการส่ง request เดิมซ้ำ เช่น กรณีอินเทอร์เน็ตขาดหายหรือผู้ใช้กดปุ่มซ้ำ
6. ให้ client สามารถตรวจสอบสถานะการจองได้อย่างชัดเจน

โปรแกรมตัวอย่างทำงานในรูปแบบ console client/server เพื่อแสดงให้เห็นว่า protocol สามารถทำงานได้ถูกต้อง

2. ลักษณะของ Application

TICKETPASS เป็นระบบจองบัตรแบบ Client-Server

- **Client** คือโปรแกรมของผู้ใช้ที่ส่งคำขอจองบัตร
- **Server** คือโปรแกรมที่เก็บจำนวนบัตร ตรวจสอบเงื่อนไข และตอบกลับผลการจอง
- Server รองรับ client หลายรายพร้อมกันโดยสร้าง thread แยกสำหรับแต่ละ connection
- ระบบเริ่มต้นมีบัตร 3 โซน ได้แก่ VIP จำนวน 5 ใบ, Zone A จำนวน 20 ใบ และ Zone B จำนวน 50 ใบ
- การจองแต่ละครั้งมีสถานะที่ชัดเจน ได้แก่ **HELD**, **BOOKED**, **CANCELLED** และ **EXPIRED**

ตัวอย่างการใช้งานจริงของแนวคิดนี้ ได้แก่ ระบบจองบัตรคอนเสิร์ต กีฬา ภาพยนตร์ งานสัมมนา เวิร์กช็อป หรือกิจกรรมที่มีจำนวนผู้เข้าร่วมจำกัด

3. Transport Layer Service Model

TICKETPASS เลือกใช้ TCP (Transmission Control Protocol)

เหตุผลที่เลือกใช้ TCP มีดังนี้

1. การจองบัตรเป็นข้อมูลสำคัญ หาก request สูญหาย ผู้ใช้อาจไม่ทราบว่าจองสำเร็จหรือไม่
2. TCP รับประกันว่าข้อมูลจะถูกส่งถึงปลายทางอย่างถูกต้องและเรียงลำดับ
3. TCP มีการตรวจสอบข้อผิดพลาดและส่งข้อมูลซ้ำเมื่อ packet สูญหาย
4. การยืนยันการจอง เช่น **CONFIRM** จำเป็นต้องถึง Server อย่างครบถ้วน
5. TCP เหมาะกับรูปแบบการสื่อสารแบบ request-response ระหว่าง client และ server

แม้ UDP จะมี overhead ต่ำกว่า แต่ UDP ไม่รับประกันว่าข้อมูลจะถึงปลายทางหรือเรียงลำดับถูกต้อง ดังนั้นจึงไม่เหมาะกับข้อมูลการจองที่ต้องการความถูกต้องสูง

4. ชื่อ Protocol

ชื่อ protocol คือ **TICKETPASS**

เป็น protocol สำหรับกำหนดรูปแบบข้อความและกติกาการจองบัตรชั่วคราวก่อนยืนยันการจองจริง

ข้อความของ protocol ใช้การเข้ารหัส UTF-8 และจบด้วยบรรทัดว่าง `\n\n`

5. รูปแบบของ Message

TICKETPASS ใช้รูปแบบข้อความแบบ text-based เพื่อให้อ่านง่าย และสามารถตรวจสอบผ่าน Wireshark ได้สะดวก

รูปแบบ request มีดังนี้

TICKETPASS/1.0 <COMMAND>

Header-Name: Header-Value

รูปแบบ response มีดังนี้

TICKETPASS/1.0 <STATUS-CODE> <STATUS-PHRASE>

Header-Name: Header-Value

6. Request Messages

6.1 CHECK

ใช้ตรวจสอบจำนวนบัตรคงเหลือในแต่ละโซน หรือระบุโซนที่ต้องการตรวจสอบ

TICKETPASS/1.0 CHECK

Zone: VIP

ตัวอย่าง response

TICKETPASS/1.0 200 OK

Zone: VIP

Available: 5

6.2 HOLD

ใช้ล็อกบัตรชั่วคราวก่อนยืนยันการจอง

TICKETPASS/1.0 HOLD

Request-ID: REQ-001

User-ID: U001

Zone: VIP

Quantity: 2

Hold-Seconds: 60

ความหมายของ header

- **Request-ID** คือรหัสเฉพาะของคำขอ ใช้ป้องกันการประมวลผล request ซ้ำ
- **User-ID** คือรหัสผู้ใช้
- **Zone** คือโซนของบัตร เช่น VIP, A หรือ B
- **Quantity** คือจำนวนบัตรที่ต้องการ
- **Hold-Seconds** คือระยะเวลาที่ล็อกบัตรไว้ชั่วคราว

ตัวอย่าง response เมื่อสำเร็จ

TICKETPASS/1.0 201 HELD

Request-ID: REQ-001

Reservation-ID: RES-AB12CD34

Zone: VIP

Quantity: 2

Expires-At: 2026-08-29T15:03:00+00:00

6.3 CONFIRM

ยืนยันการจองหลังจากผู้ใช้ดำเนินการชำระเงินแล้ว

TICKETPASS/1.0 CONFIRM

Reservation-ID: RES-AB12CD34

ตัวอย่าง response

TICKETPASS/1.0 200 BOOKED

Reservation-ID: RES-AB12CD34

Zone: VIP

Quantity: 2

6.4 CANCEL

ยกเลิกการจองที่ยังอยู่ในสถานะ **HELD** ระบบจะคืนจำนวนบัตรกลับเข้า inventory

TICKETPASS/1.0 CANCEL

Reservation-ID: RES-AB12CD34

ตัวอย่าง response

TICKETPASS/1.0 200 CANCELLED

Reservation-ID: RES-AB12CD34

Status: CANCELLED

6.5 STATUS

ใช้ตรวจสอบสถานะของ reservation

TICKETPASS/1.0 STATUS

Reservation-ID: RES-AB12CD34

ตัวอย่าง response

TICKETPASS/1.0 200 OK

Reservation-ID: RES-AB12CD34

Status: HELD

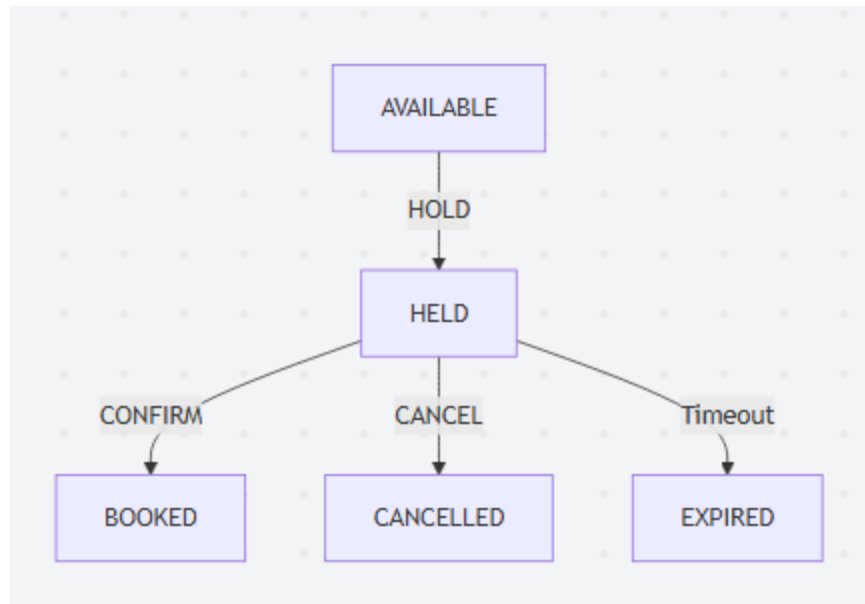
Zone: VIP

Quantity: 2

7. Response Status Codes

Status Code	Status Phrase	ความหมาย
200	OK	คำขอสำเร็จ
200	BOOKED	ยืนยันการจองสำเร็จ
200	CANCELLED	ยกเลิกการจองสำเร็จ
201	HELD	ล็อกบัตรชั่วคราวสำเร็จ
400	BAD_REQUEST	รูปแบบข้อความไม่ถูกต้อง หรือข้อมูลไม่ครบ
404	NOT_FOUND	ไม่พบโซนบัตรหรือ Reservation-ID
409	SOLD_OUT	บัตรที่ต้องการมีไม่เพียงพอ
410	HOLD_EXPIRED	การล็อกบัตรหมดอายุแล้ว

8. Reservation State Model



เมื่อ reservation อยู่ในสถานะ **HELD** จำนวนบัตรจะถูกตัดออกจาก inventory ชั่วคราว หากมีการ **CONFIRM** จะเปลี่ยนเป็น **BOOKED** แต่หากผู้ใช้ **CANCEL** หรือหมดเวลาที่กำหนด บัตรจะถูกคืนกลับเป็น **AVAILABLE**

9. จุดเด่นของ TICKETPASS

9.1 ป้องกันการจองเกินจำนวน

Server ใช้ lock ระหว่างตรวจสอบและปรับจำนวนบัตร ทำให้การจองเป็น atomic operation แม้ client หลายคนจะส่ง **HOLD** เข้ามาพร้อมกัน จำนวนบัตรรวมที่ถูกล็อกจะไม่เกินจำนวนจริง

ตัวอย่าง: หากโซน VIP เหลือ 5 ใบ และ Client 1 ขอ 3 ใบพร้อมกับ Client 2 ขอ 3 ใบ Client แรกจะได้รับ **201 HELD** ส่วนอีก client จะได้รับ **409 SOLD_OUT**

9.2 การล๊อบบัตรชั่วคราว

หลังได้รับ **HOLD** ผู้ใช้มีเวลาในการยืนยันการจองตามค่า **Hold-Seconds** หากไม่ยืนยันภายในเวลา ระบบจะเปลี่ยนสถานะเป็น **EXPIRED** และคืนบัตรสู่ระบบ

9.3 ป้องกัน Request ซ้ำ

TICKETPASS ใช้ **Request-ID** ในคำสั่ง **HOLD** หาก client ส่ง request เดิมซ้ำ Server จะส่งผลลัพธ์เดิมกลับไป โดยไม่ตัดจำนวนบัตรเพิ่ม

คุณลักษณะนี้เรียกว่า **Idempotency** ซึ่งมีประโยชน์เมื่อผู้ใช้กดปุ่มจองซ้ำ หรือไม่แน่ใจว่า request เติมส่งถึง server แล้วหรือไม่

9.4 มีสถานะการจองที่ชัดเจน

Client สามารถใช้คำสั่ง **STATUS** เพื่อตรวจสอบได้ว่า reservation อยู่ในสถานะ **HELD**, **BOOKED**, **CANCELLED** หรือ **EXPIRED** ลดความกำกวมว่าการจองสำเร็จหรือไม่

10. ความแตกต่างจากระบบทั่วไป

HTTP เป็น protocol สำหรับรับส่ง request และ response ทั่วไป แต่ไม่ได้กำหนดความหมายเฉพาะสำหรับการล็อกบัตร, เวลาหมดอายุ, การยืนยันการจอง หรือ การป้องกัน request ซ้ำ

TICKETPASS เป็น application-layer protocol เฉพาะทางที่กำหนดคำสั่งและสถานะสำหรับการจองบัตรโดยตรง เช่น **HOLD**, **CONFIRM**, **CANCEL**, **STATUS**, **HELD** และ **HOLD_EXPIRED**

ดังนั้นจุดเด่นของ TICKETPASS คือ **Reservation Correctness** หรือความถูกต้องของกระบวนการจอง ภายใต้สถานการณ์ที่บัตรมีจำนวนจำกัดและผู้ใช้จำนวนมาก อาจจองพร้อมกัน

11. การทดสอบด้วย Wireshark

สามารถตรวจสอบการทำงานของ protocol ด้วย Wireshark ได้ โดย capture ที่ interface Loopback เนื่องจาก client และ server ทำงานบนเครื่องเดียวกัน

ใช้ Display Filter ดังนี้

```
tcp.port == 5050 && tcp.len > 0
```

จากนั้นเลือก packet ที่มีข้อมูลและคลิกขวา เลือก

Follow → TCP Stream

Wireshark จะแสดงข้อความ TICKETPASS request และ response ที่ถูกส่งจริงผ่าน TCP เช่น **HOLD** และ **201 HELD** หรือ **HOLD** และ **409 SOLD_OUT**

12. สรุป

TICKETPASS เป็น protocol สำหรับระบบจองบัตรคอนเสิร์ตที่ออกแบบมาเพื่อรองรับบัตรจำนวนจำกัด โดยมีความสามารถในการล็อกบัตรชั่วคราว ยืนยันหรือยกเลิกการจอง ตรวจสอบสถานะ และป้องกันการจองเกินจำนวนเมื่อมี client หลายรายใช้งานพร้อมกัน

Protocol นี้เลือกใช้ TCP เพื่อให้การส่งข้อมูลการจองมีความน่าเชื่อถือและเรียงลำดับถูกต้อง จุดเด่นสำคัญคือการกำหนดกติกาการจองที่ชัดเจน ทำให้ client และ server เข้าใจสถานะการจองตรงกัน และช่วยลดปัญหา overselling ในระบบจองบัตรครับ

